



UI自动化测试

鸿蒙自动化



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌



鸿蒙自动化

DevEco Testing Hypium



DevEco Testing Hypium(以下简称**Hypium**)是HarmonyOS平台的UI自动化测试框架,支持开发者使用python语言为应用编写UI自动化测试脚本,主要包含以下特性:

- Hypium提供了原生**控件/图像/比例坐标**等多种控件定位能力,支持多窗口操作以及触摸屏/鼠标/键盘等多种模拟输入功能,支持多设备并行操作,能够覆盖各类场景和多种形态设备上的自动化用例编写需求。
- Hypium包含配套**用例编写辅助插件**,支持控件查看/投屏操作等多种用例开发辅助功能,提升用例开发体验和效率。
- Hypium能够为执行的用例生成详细的**用例执行报告**,并且自动记录设备日志以及执行步骤截图,为开发者和测试人员提供高效和专业的测试用例执行和结果分析体验。

鸿蒙自动化测试环境搭建步骤



python: 解释器
pycharm: 编码工具

hdc: 电脑与手机调
试工具

Hypium提供自动化实现的
常用方法

获取元素基本信息

Hypium安装



Hypium: 鸿蒙UI自动化测试框架库，里面提供各种操作UI的API如查找、点击、输入等。

- 下载Hypium安装包: <https://developer.huawei.com/consumer/cn/download/>
- 解压Hypium安装包: 解压该文件后得到4个tar.gz格式的pip安装包
- 安装Hypium: 使用pip install命令安装，Hypium安装对xdevice有依赖，**优先安装xdevice**

```
pip install xdevice-5.0.7.200.tar.gz
pip install xdevice-devicetest-5.0.7.200.tar.gz
pip install xdevice-ohos-5.0.7.200.tar.gz
pip install hypium-5.0.7.200.tar.gz
```

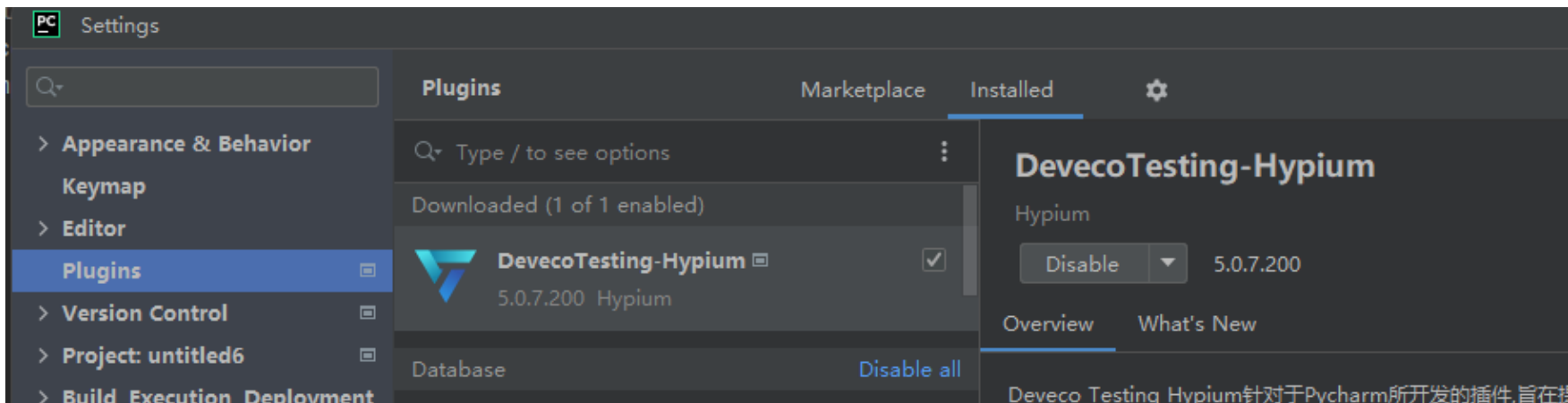
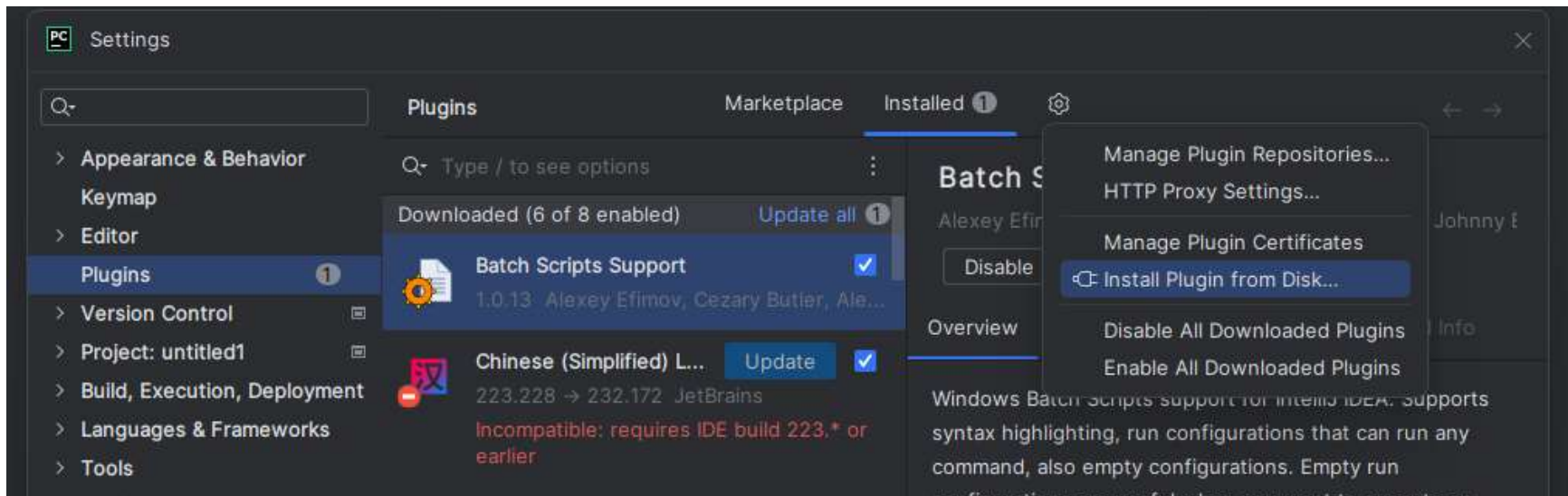
```
C:\Users\Jack>pip show hypium
Name: hypium
Version: 5.0.7.200
Summary: 鸿蒙测试框架
Home-page:
Author:
Author-email:
License:
Location: C:\python311\Lib\site-packages
Requires: xdevice, xdevice-devicetest, xdevice-ohos
Required-by:
```

元素定位工具插件UIViewer安装

- 作用：查看元素信息便于自动化hypium api通过元素信息来操作元素实现自动化
- 工具：DevecoTesting-Hypium-5.0.7.200.zip（Hypium下载包中已包含）
- 安装：
 - 打开pycharm后，点击File -> Settings -> Plugin -> 齿轮图标 -> **Install Plugin from Disk** -> 选中下载的离线安装zip包 -> 安装完成后**重启pycharm**。



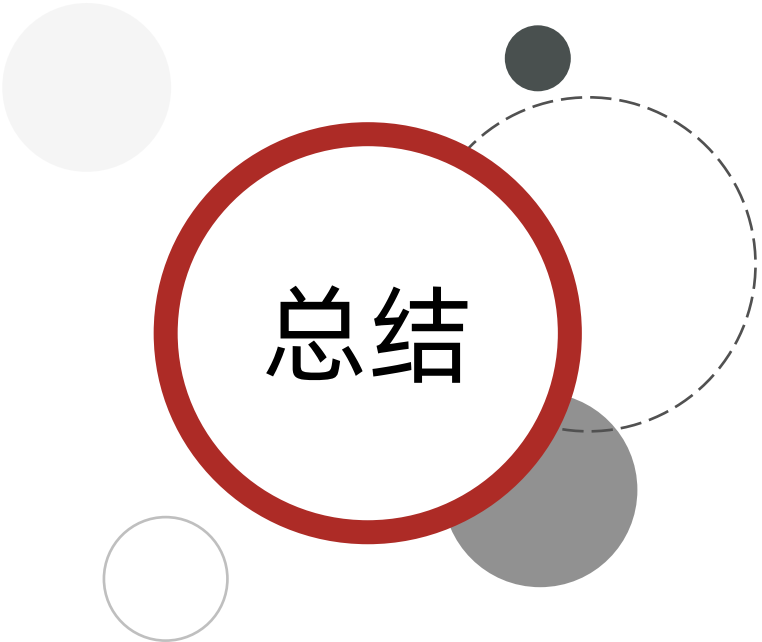
元素定位工具插件UIViewer安装





练习

Hypium自动化环境搭建



总结

1. 如何搭建Hypium自动化测试环境?

- ✓ 安装python
- ✓ 安装Pycharm
- ✓ 安装hdc
- ✓ 安装Hypium
- ✓ 安装UIViewer插件



Hypium入门案例



启动系统设置程序

需求：自动启动系统设置程序

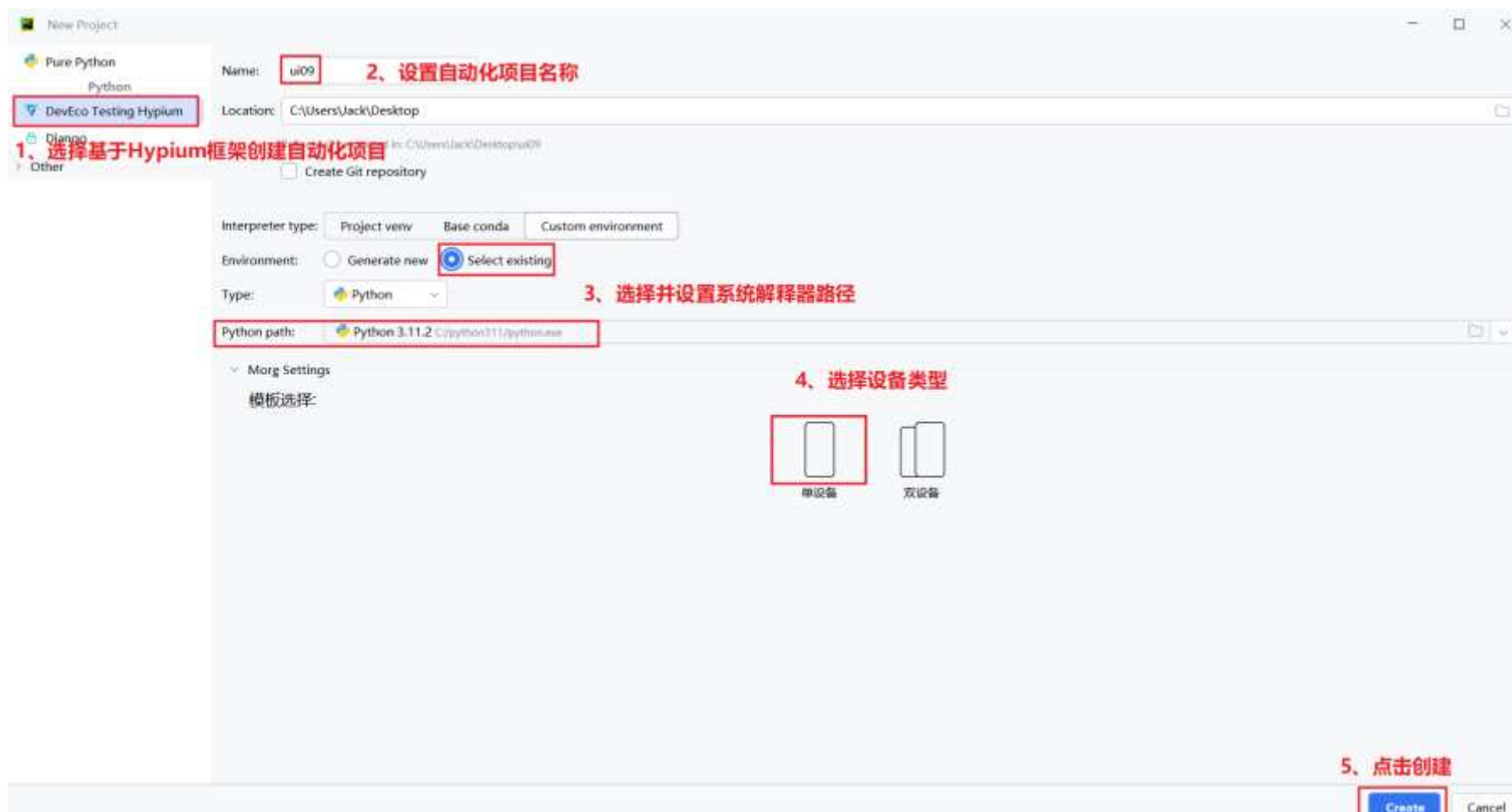
步骤：

- 1、创建自动化项目
- 2、编写自动化脚本
- 3、运行自动化脚本

1、创建自动化项目



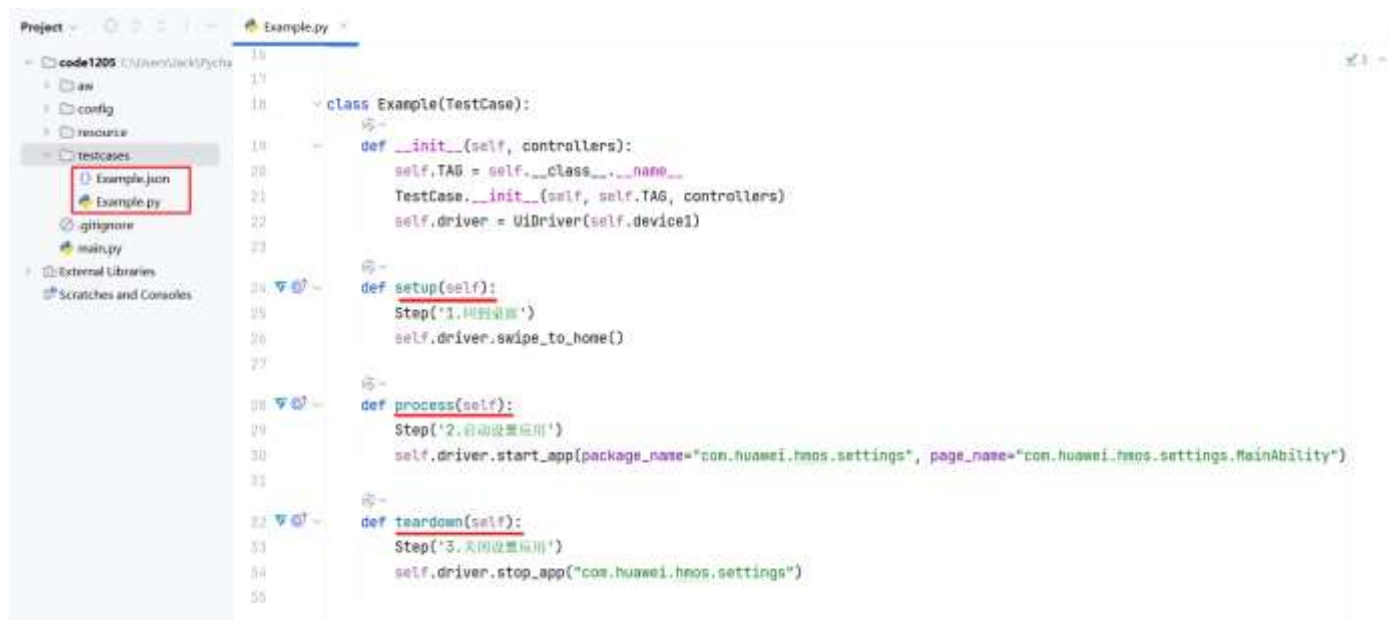
- pycharm: 点击File → new project → 选择DevEco Testing Hypium创建项目



2、编写自动化脚本



- 项目工程其中包含了一个模板用例和一个模板user_config.xml

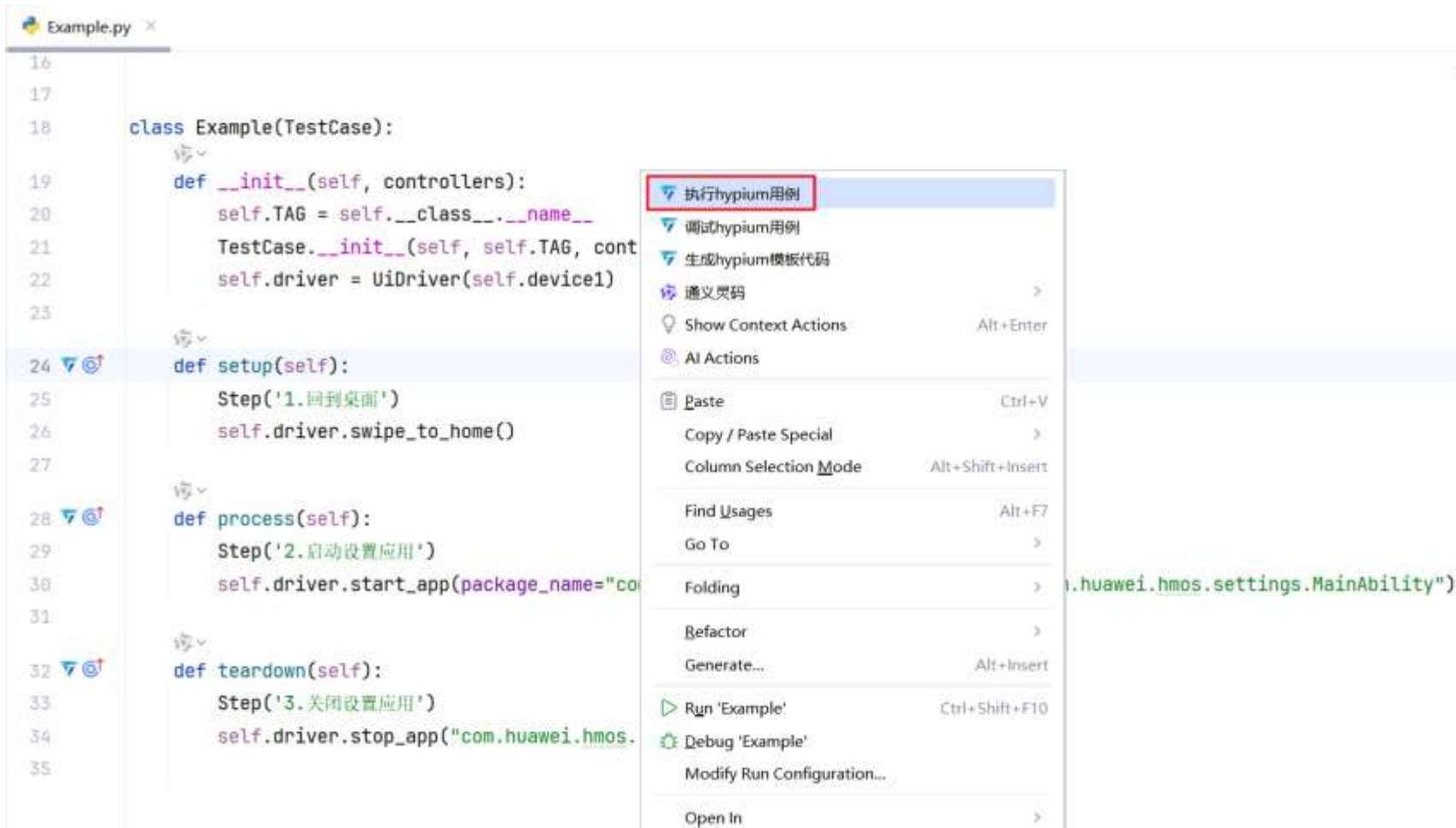


序号	生命周期函数	说明
1	setup	测试用例的前置步骤，主要用于执行测试用例的预置动作
2	process	测试用例的实际操作步骤，主要描述当前测试用例中的所有测试用例步骤集合
3	teardown	测试用例的清理步骤，主要用于执行测试用例的环境清理等操作

3、运行自动化脚本



- 右键 → 执行Hypium用例



总结

1、自动化测试用例核心代码编写是在哪里？

2、如何启动指定的应用程序？

3、扩展：

```
HypiumProjectTemplate
| |--aw                      // 工程中自定义模块文件夹
| | |--Utils.py             // 示例模块文件
| |--config                 // 测试工程配置文件夹
| | |--user_config.xml      // 测试工程配置文件，主要是测试框架的任务配置
| |--resource              // 测试资源文件夹，测试过程中用到的资源文件默认会优先从当前文件夹进行查找
| |--testcases             // 测试用例文件夹，测试过程中的测试用例文件优先会从当前文件夹进行查找
| | |--Example.json         // Example测试用例配置文件，配置用例设备信息等
| | |--Example.py           // Example测试用例文件，实际的测试逻辑代码
| |--main.py               // 测试用例执行入口
```



Hypium API介绍



Hypium测试框架提供了两大类API来支持用例的编写。
第一类是需要被测设备参数执行的API，第二类是无需被测设备，在PC端可独立调用的API。

- **设备相关API:**

- ✎ UiDriver
- ✎ BY
- ✎ UiComponent
- ✎ UiWindow

- **设备无关API:**

- ✎ host
- ✎ CV

设备相关的API主要包括四个基础API类：UiDriver，BY，UiComponent，UiWindow

- **UiDriver** 类为UI测试的入口，代表了一个被测设备，提供*控件查找、控件检查、用户操作模拟、执行shell命令、安装卸载应用*等等Ui测试核心能力。
- **BY**对象用于描述需要操作的*控件属性，实现控件定位*。
- **UiComponent** 为UiDriver查找返回的控件对象，提供*控件属性查询、控件点击、滑动查找等触控/检视能力*。
- **UiWindow** 为UiDriver查找返回的窗口对象，提供*窗口属性查询、窗口拖动、大小调整等触控能力*。

示例代码：

```
from hypium import UiDriver, BY, UiComponent, UiWindow
from hypium.model import WindowFilter

# 创建driver对象 (self.device1对象在测试用例类中提供)
driver = UiDriver(self.device1)

# 查找控件
component = driver.find_component(BY.text("蓝牙"))
# 点击组件
component.click()

# 查找窗口
window = driver.find_window(WindowFilter().bundle_name("com.huawei.hmos.settings"))
# 获取窗口对应的应用包名
window.getBundleName()
```

设备无关的API当前主要包括两个基础API类: host 和 CV

- **host** 提供基础值断言, PC端shell命令执行等PC端基础操作能力。
- **CV** 提供图像查找, 图像比较, 压缩, 清晰度计算等基础图像操作能力。

示例代码

```
from hypium import host, CV
```

执行PC端命令

```
echo = host.shell("a.bat")
```

调用图像接口

```
brightness = CV.calculate_brightness("/path/to/image.jpeg")
```

- 此外Hypium定义的一些常量类型，例如 *KeyCode*, *UiParam*, *MatchPattern*等，以及数据类型 *Point*, *Rect*等，这部分包含在hypium.model包中。

示例代码

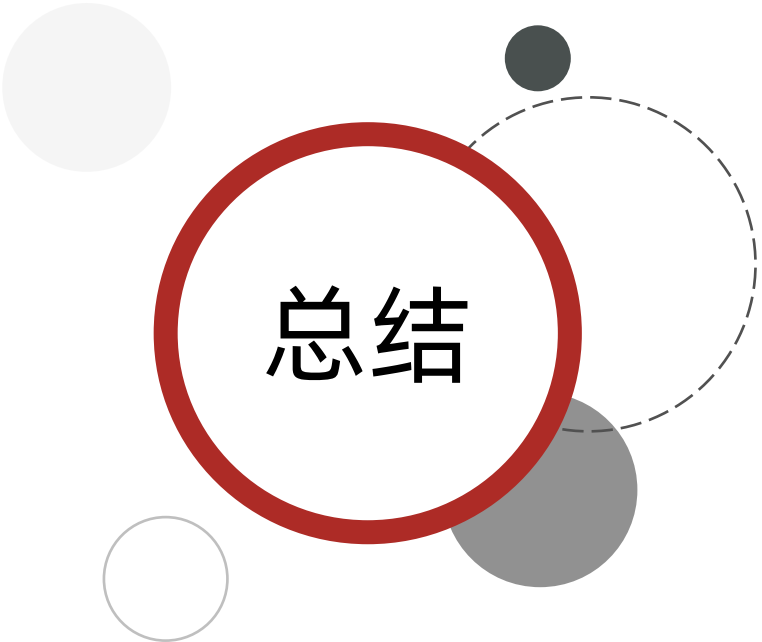
```
from hypium.model import KeyCode, UiParam, MatchPattern
```

按下电源键（使用常量KeyCode.POWER）

```
driver.press_key(KeyCode.POWER)
```

向左滑动屏幕（使用常量UiParam.LEFT）

```
driver.swipe(UiParam.LEFT)
```



总结

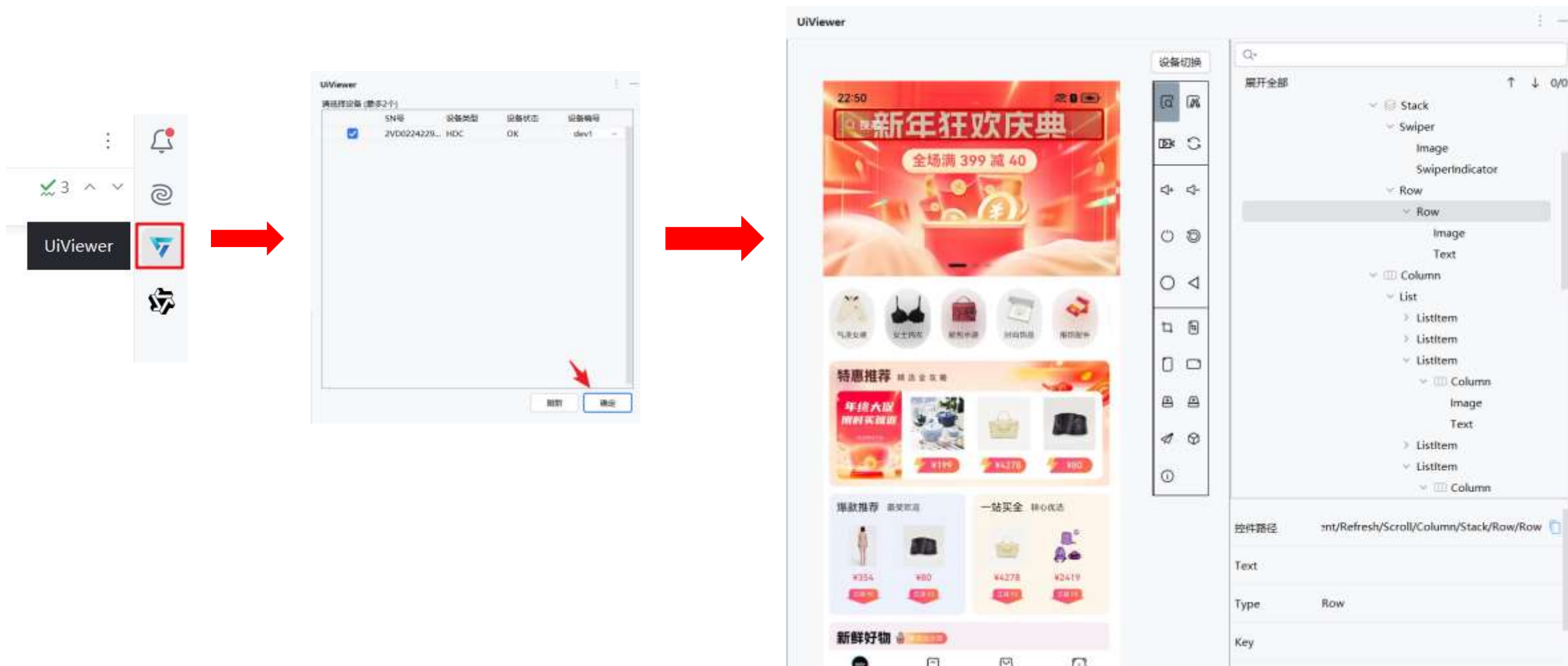
1、Hypium测试框架提供了哪两大类API来支持用例编写？

- 设备相关API: UiDriver, BY, UiComponent, UiWindow
- 设备无关API: host 和 CV



控件定位

使用DevEco Testing Hypium插件中的 **UiViewer** 工具即可查看控件的各种属性

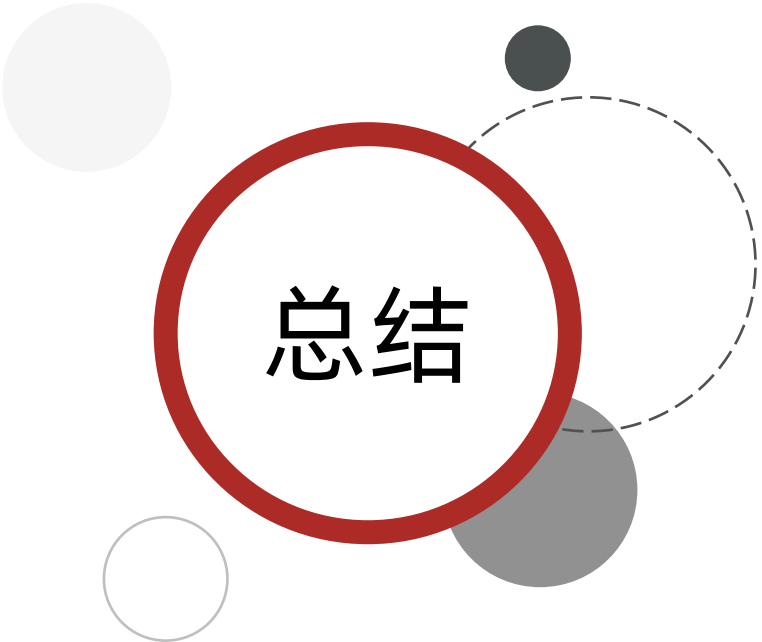


Hypium中的定位操作目标的方式主要分三大类型：

- 控件属性定位
- 图片匹配定位
- 比例坐标定位

说明：

- 根据操作目标的定位准确性，首选方式为**控件属性定位**，次选**图片匹配定位**。当无法使用前两类方式定位时，可以选择**比例坐标定位**操作目标。
- Hypium中的控件属性定位通过 **BY选择器** 对象来实现



总结

1、如何获取控件属性？

2、Hypium常见定位方式？

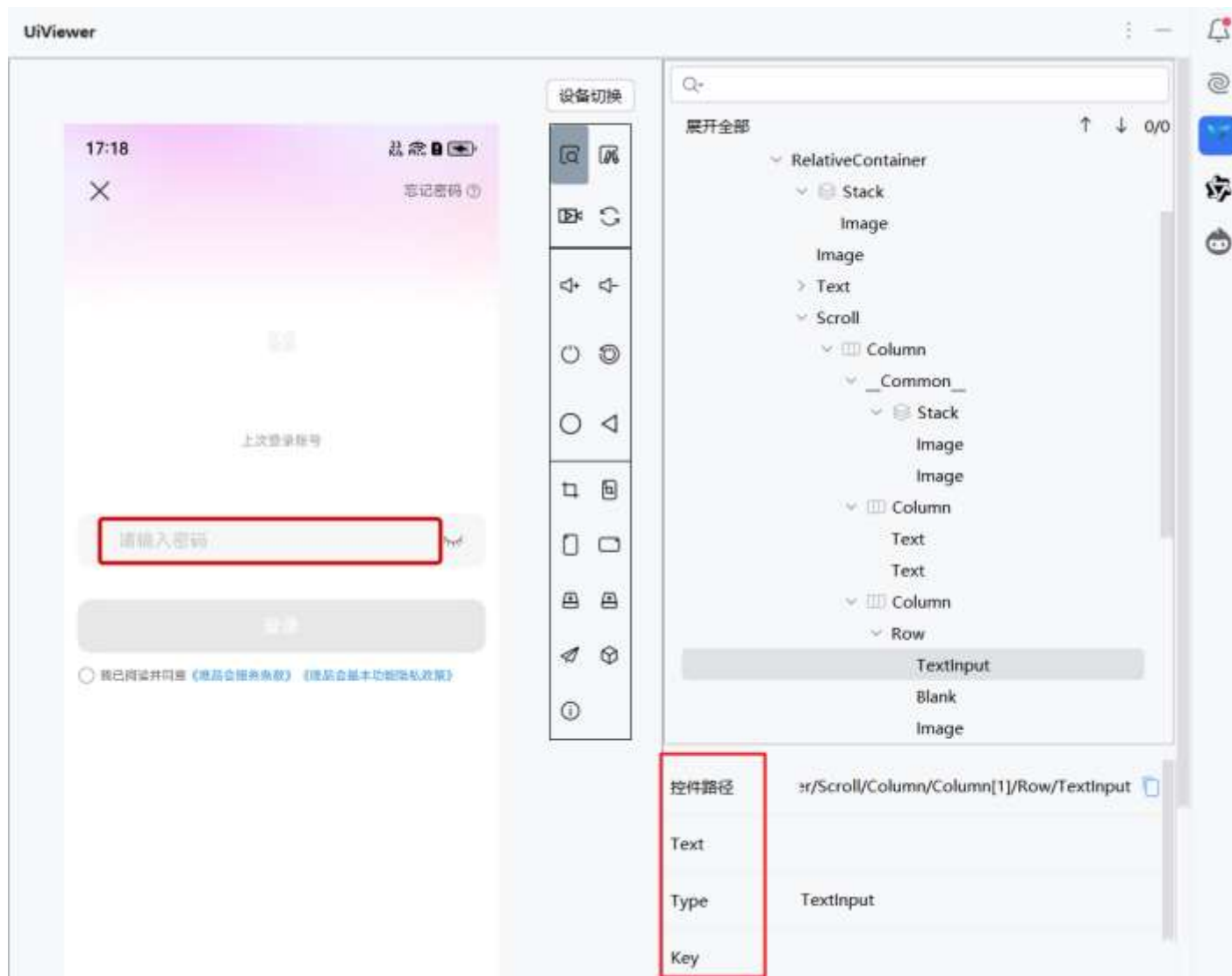
- 属性定位
- 图片定位
- 比例坐标定位



控件属性定位

- text属性定位

通过UIViewer工具可以获取到控件各种属性（如：text、key、type等）用于操作控件



Hypium中的控件属性定位通过 **BY选择器** 对象来实现，提供了如图各种属性定位方法

序号	属性名称	属性值类型	对应BY选择器	是否支持模糊匹配
1	text	str	BY.text	是
2	key	str	BY.key	否
3	type	str	BY.type	否
4	checkable	bool	BY.checkable	否
5	longClickable	bool	BY.longClickable	否
6	clickable	bool	BY.clickable	否
7	scrollable	bool	BY.scrollable	否
8	enabled	bool	BY.enabled	否
9	focused	bool	BY.focused	否
10	selected	bool	BY.selected	否
11	checked	bool	BY.checked	否

使用方法：

- 从hypium包中导入BY选择器对象, 从hypium.model包中导入匹配模式常量类MatchPattern

```
from hypium import BY
from hypium.model import MatchPattern
```

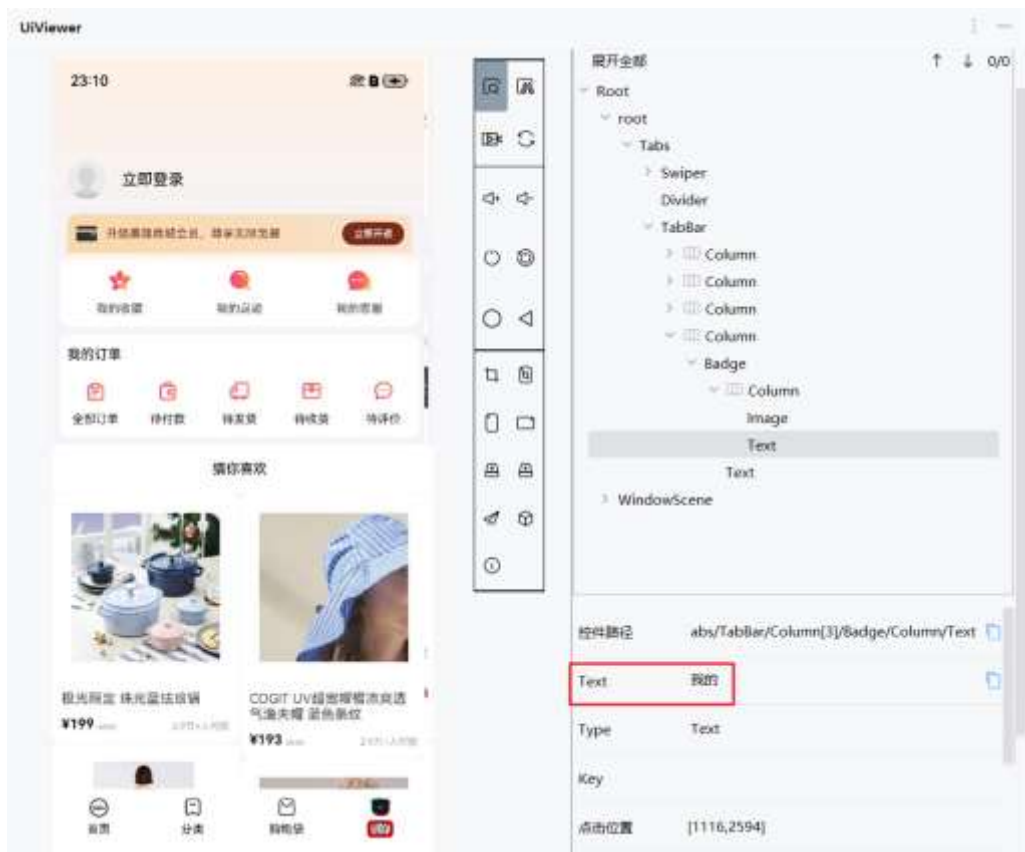
- 通过BY对象可以指定需要查找/操作的控件对象

```
# 查找text属性为"控件文本"的控件
component = driver.find_component(BY.text("蓝牙"))
# 直接点击控件
component = driver.touch(BY.text("蓝牙"))
```

说明：默认情况下，find_component 和 touch等方法会 **查找/操作第一个条件匹配的控件**，如需操作第n个满足匹配条件的控件，请参考查找所有匹配控件。

文本属性定位：通过UIViewer获取到控件的 **text** 属性值进行控件定位

- 定位方法：driver.find_component(BY.text("text属性值"))
- 示例：driver.find_component(BY.text("我的"))

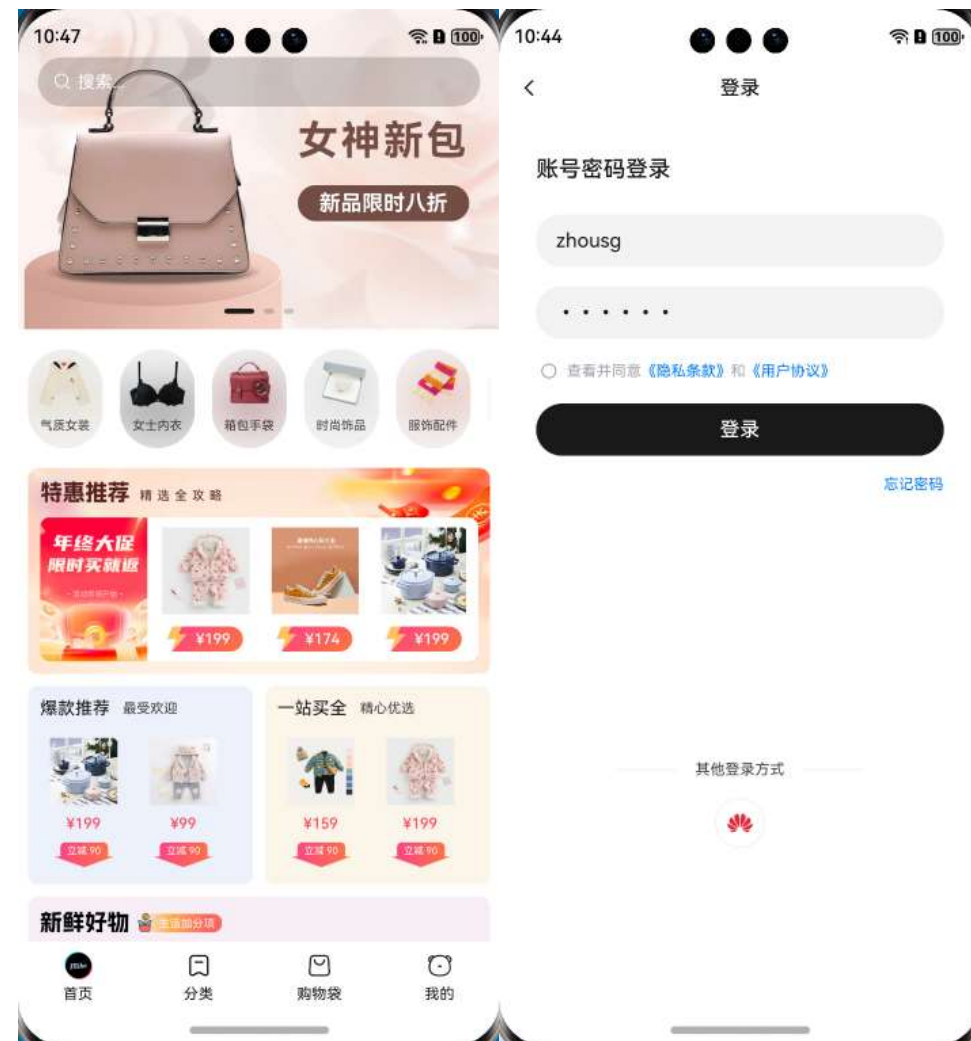


案例

启动商城APP并打开账号密码登录页面

分析：

- 1、启动商城APP
- 2、点击【我的】进入个人中心
- 3、点击【立即登录】打开登录页



示例代码：

```
Step("1、返回桌面")
self.driver.swipe_to_home()

Step("2、启动APP")
# 使用aa命令获取包名：hdc shell aa dump -l
# 获取前先启动对用app
# 美寇商城：(com.itcast.mk_shop, EntryAbility)
self.driver.start_app("com.itcast.mk_shop")

Step("3、点击我的进入个人中心页")
self.driver.find_component(BY.text("我的")).click()

Step("4、点击立即登录进入登录页")
self.driver.find_component(BY.text("立即登录")).click()
```

模糊匹配：

当前控件的text属性支持三种 模糊匹配方式，通过BY.text方法第二个可选参数指定，如下表所示：

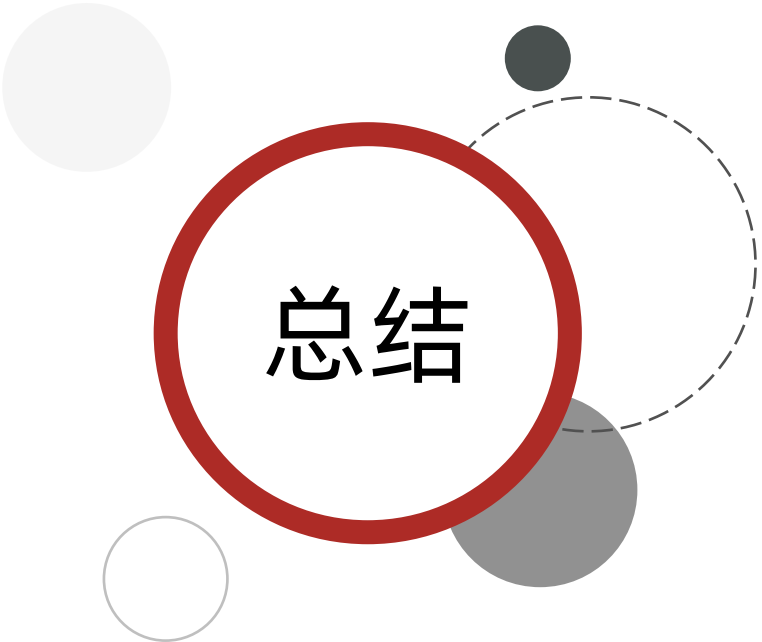
序号	匹配方式常量	说明
1	MatchPattern.STARTS_WITH	前缀匹配
2	MatchPattern.ENDS_WITH	后缀匹配
3	MatchPattern.CONTAINS	包含匹配

示例代码：

```
# 点击text属性以`立即`开头的控件
driver.touch(BY.text("立即", MatchPattern.STARTS_WITH))

# 点击text属性以`登录`结尾的控件
driver.touch(BY.text("登录", MatchPattern.ENDS_WITH))

# 点击text属性包含`登`的控件
driver.touch(BY.text("登", MatchPattern.CONTAINS))
```



总结

1、如何基于控件文本属性定位及操作控件？

- **定位：**

- 精确匹配： `component = driver.find_component(BY.text("文本属性值"))`
- 模糊匹配：
 - 前缀匹配： `driver.touch(BY.text("文本属性开始值", MatchPattern.STARTS_WITH))`
 - 后缀匹配： `driver.touch(BY.text("文本属性结尾值", MatchPattern.ENDS_WITH))`
 - 包含匹配： `driver.touch(BY.text("文本属性包含值", MatchPattern.CONTAINS))`

- **操作：**

- 点击： `component.click()`



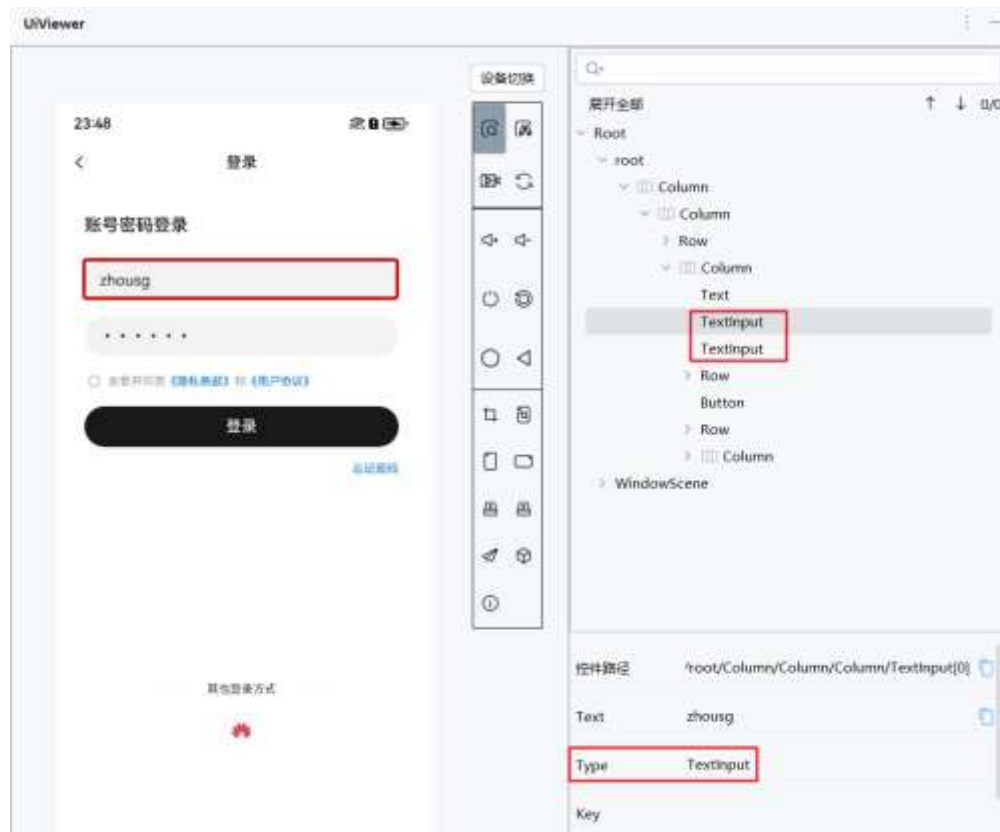
控件属性定位

- type属性定位

type属性定位：通过UIViewer获取到控件的 **type** 属性值进行控件定位

- 定位方法：

- 定位一组控件：driver.find_all_components(BY.type("type属性值"))
- 说明：返回值为控件**列表**，可以使用**下标**选择指定的控件
- 示例：driver.find_all_components(BY.type("TextInput"))



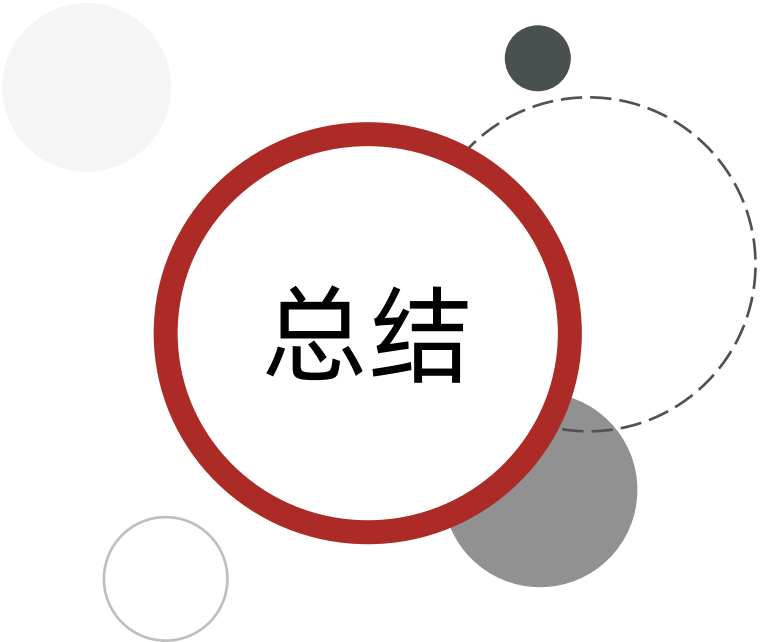
案例

自动登录



示例代码：

```
# 清空账号输入框
self.driver.find_all_components(BY.type("TextInput"))[0].clearText()
# 输入账号信息
self.driver.find_all_components(BY.type("TextInput"))[0].inputText("zhousg")
# 清空密码输入框
self.driver.find_all_components(BY.type("TextInput"))[1].clearText()
# 输入密码信息
self.driver.find_all_components(BY.type("TextInput"))[1].inputText("123456")
# 勾选协议
self.driver.find_component(BY.type("Checkbox")).click()
# 点击登录
self.driver.find_component(BY.type("Button")).click()
```



总结

1、如何基于控件type属性定位及操作控件？

- **定位：** `driver.find_all_components(BY.type("type属性值"))`
- **操作：**
 - 清空： `component.clearText()`
 - 输入： `component.inputText("输入数据")`



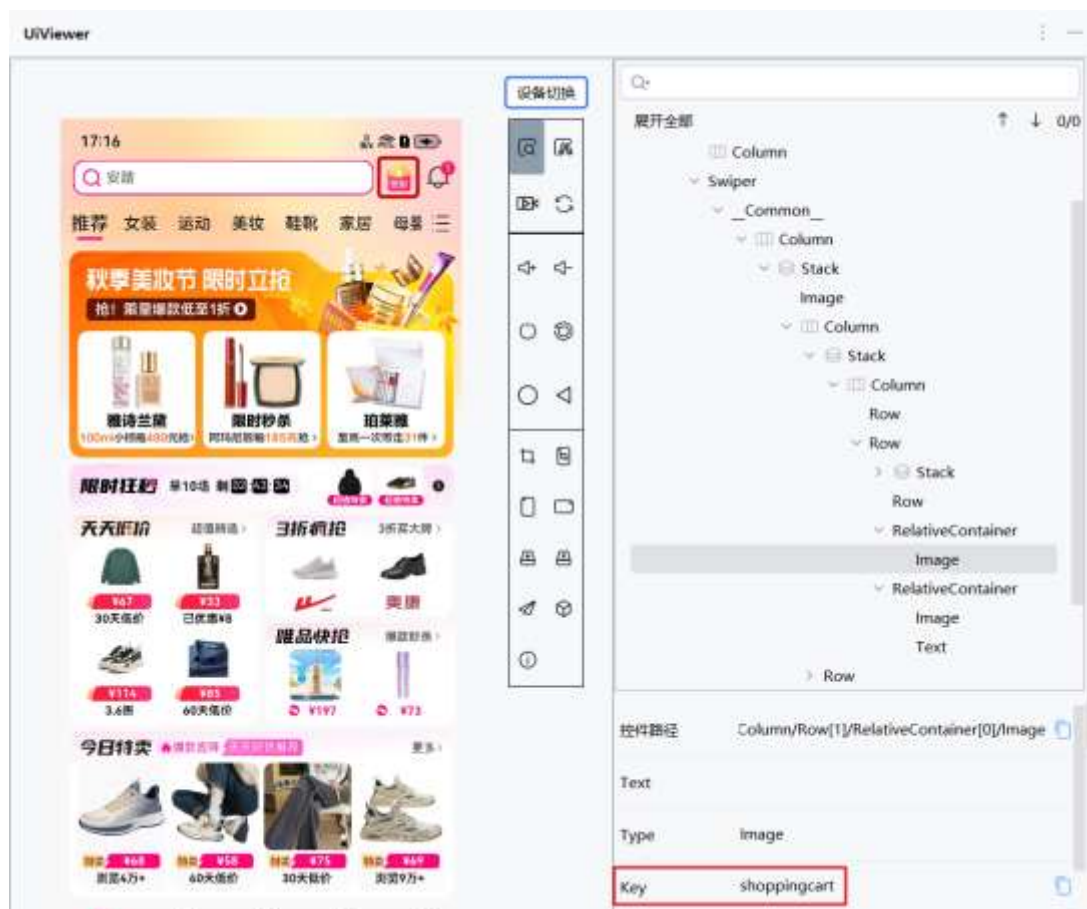
控件属性定位

- key属性定位

key属性定位：通过UIViewer获取到控件的 **key** 属性值进行控件定位

- 定位方法：

- 定位一组控件：driver.find_component(BY.key("key属性值"))
- 示例：driver.find_component(BY.key("shoppingcart"))



案例

自动签到



示例代码:

```
# 已登录
```

```
Step("预置条件1: 启动APP")
```

```
# ('com.vip.hosapp', 'EntryAbility')
```

```
self.driver.start_app("com.vip.hosapp")
```

```
Step("步骤1: 进入签到页面")
```

```
self.driver.find_component(BY.key("shoppingcart")).click()
```

```
time.sleep(3)
```

```
Step("步骤2: 点击签到")
```

```
self.driver.find_component(BY.text("立即签到")).click()
```

```
time.sleep(3)
```

```
Step("收尾工作1: 关闭app")
```

```
self.driver.stop_app("com.vip.hosapp")
```

总结

1、如何基于控件key属性定位控件？

- **定位：** `driver.find_components(BY.key("key属性值"))`

2、扩展：常见属性定位

序号	属性名称	属性值类型	对应BY选择器	是否支持模糊匹配
1	text	str	BY.text	是
2	key	str	BY.key	否
3	type	str	BY.type	否
4	checkable	bool	BY.checkable	否
5	longClickable	bool	BY.longClickable	否
6	clickable	bool	BY.clickable	否
7	scrollable	bool	BY.scrollable	否
8	enabled	bool	BY.enabled	否
9	focused	bool	BY.focused	否
10	selected	bool	BY.selected	否
11	checked	bool	BY.checked	否



08

控件相对位置+属性组合定位

- 多属性组合定位
- 控件相对位置+属性组合定位

BY选择器支持使用链式调用的方式指定多个属性来定位一个控件，在多个控件有部分属性相同，部分属性不同时可以更精确地定位控件。

```
# 点击文本为"蓝牙", 类型为"Button", 并且key为"bluetooth_switch"的按钮
```

```
driver.touch(BY.text("蓝牙").type("Button").key("bluetooth_switch"))
```

```
# 同样，在查找以及其他可以传入BY对象的接口中可以使用相同的用法
```

```
component = driver.find_component(BY.text("蓝牙").type("Button").key("bluetooth_switch"))
```

对于某些自身属性不唯一，无法精确定位的控件，BY选择器支持通过与其他控件的相对位置关系来提高定位控件的精确性。支持的相对定位方式如下表所示。

序号	相对位置定位接口	功能描述
1	BY.isBefore	匹配在指定控件前边的控件
2	BY.isAfter	匹配在指定控件后边的控件
3	BY.within	匹配在指定控件内部的控件
4	BY.inWindow	匹配在指定控件内部的控件 (在支持多窗口的设备中通过包名指定控件所在的窗口，不属于控件相对位置)

案例 点击显示通知图标之后的按钮



分析：

分析：

1. 首先选择一个可以通过属性唯一定位的锚点控件。例如BY.text("显示通知图标")
2. 然后找到想要操作的目标控件，选择该控件的一个不唯一属性(通常为type属性)。例如BY.type("Button")
3. 使用相对位置接口来描述锚点控件和目标控件的位置的关系，得到完整的控件选择器。例如BY.type("Button").isAfter(BY.text("显示通知图标")), 注意到这里组合使用了type属性和isAfter相对位置接口。

示例：

```
# 查找在text属性为"显示通知图标"的控件之后的type属性为"Button"的控件
component = driver.find_component(BY.type("Button").isAfter(BY.text("显示通知图标")))
```

总结

1、对于某些自身属性不唯一，无法精确定位的控件如何定位？

- 多属性组合定位
- 控件相对位置+属性组合定位控件

序号	相对位置定位接口	功能描述
1	BY.isBefore	匹配在指定控件前边的控件
2	BY.isAfter	匹配在指定控件后边的控件
3	BY.within	匹配在指定控件内部的控件
4	BY.inWindow	匹配在指定控件内部的控件 (在支持多窗口的设备中通过包名指定控件所在的窗口， 不属于控件相对位置)

- **注意：**相对位置中的锚点控件不能再使用相对位置描述，即BY.isBefore方法的参数中不能出现BY.isBefore或者BY.isAfter等相对定位的方式。

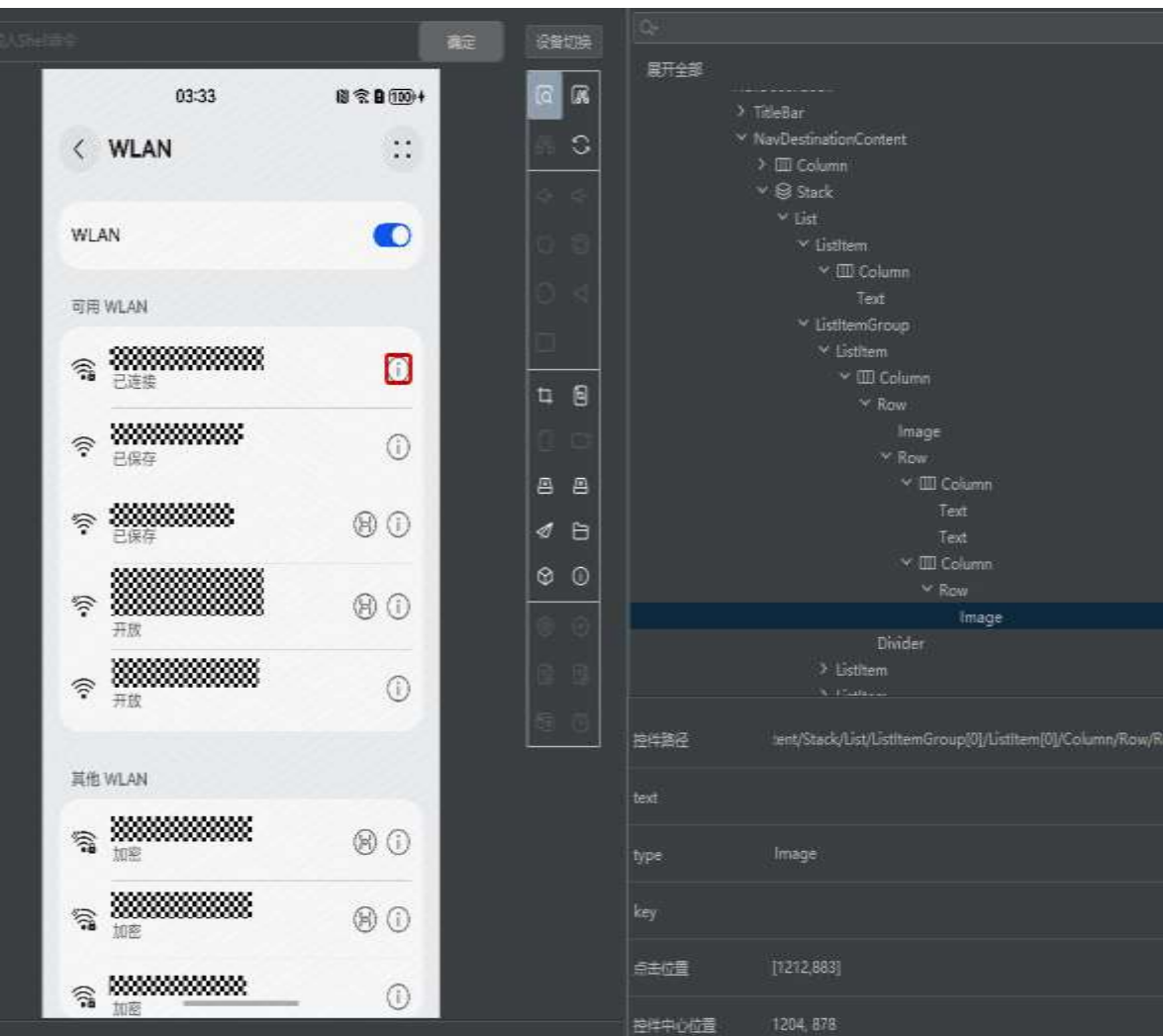


xpath定位

- 场景：部分控件没有唯一定位的属性，同时通过相对定位的方式也无法准确定位，此时可以使用xpath语法来进行更新精确的控件定位。
- 说明：使用BY.xpath匹配器可以支持通过xpath语法来查找控件。注意xpath不能和其他匹配器一起使用，并且通过xpath查找控件会慢一些。

案例

定位到红框标识的图标



分析:

该页面上, "可用 WLAN"是一个固定的可唯一定位的文本, 我们首先通过xpath定位到该文本 `//*[text()='可用 WLAN']`, 然后在找这个节点所在的List控件/`ancestor::List`, 然后从这个List控件开始找到对应的Image控件/`ListItemGroup/ListItem[1]/Text/following::Image`。

示例:

查找上图中红框所示的图标, 并点击

```
comp = driver.find_component(BY.xpath("//*[text()='可用  
WLAN']/ancestor::List/ListItemGroup/ListItem[1]/Text/following::Image"))  
comp.click()
```

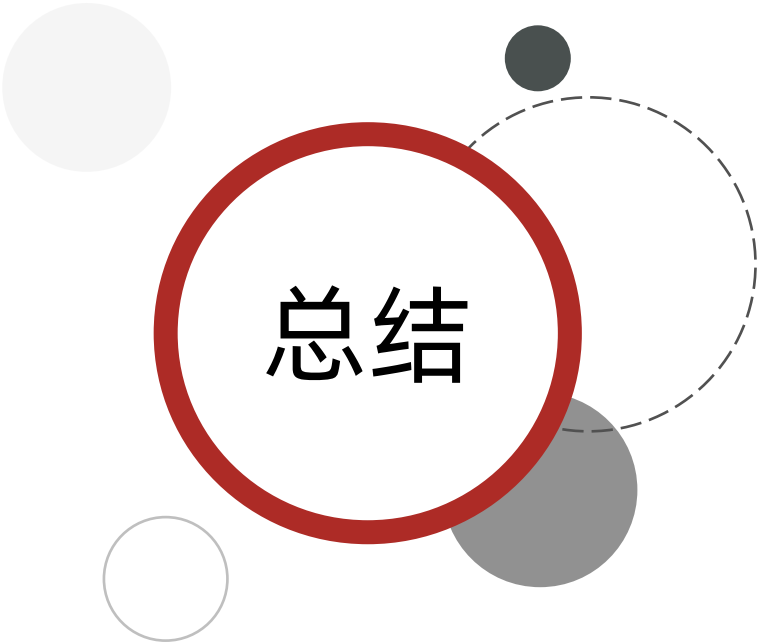
说明: 在支持传入BY选择器的接口上都可以使用xpath来定位控件

查找text属性为WLAN的控件

```
driver.find_component(BY.xpath("//*[text()='WLAN']"))  
driver.find_all_components(BY.xpath("//*[text()='WLAN']"))  
driver.wait_for_component(BY.xpath("//*[text()='WLAN']"))
```

点击text属性为WLAN的控件

```
driver.touch(BY.xpath("//*[text()='WLAN']"))
```



总结

1、部分控件没有唯一定位的属性，同时通过相对定位的方式也无法准确定位？

- xpath定位



图片定位控件

场景：当控件没有可以用于定位的唯一属性时，可以通过截取控件图片信息进行定位

- 例如：红框中的控件没有可供唯一定位的属性，同时附近的控件也无法唯一定位导致不能使用相对位置定位时，可以尝试使用图片匹配的方案定位控件。



- 使用：

- 控件查找：driver.find_image("图片信息")
- 控件操作：driver.touch_image("图片信息")

- 示例：

```
# 点击屏幕上和模板图片template.jpeg匹配的位置  
driver.touch_image("/path/to/template.jpeg")  
  
# 查找屏幕上和模板图片template.jpeg匹配的位置  
driver.find_image("template.jpeg")
```

- 注意：

- 使用图片定位控件必须安装opencv-python包（命令：***pip install opencv-python***）
- 当前仅支持查找匹配度最高的第一个匹配的图片区域，不支持匹配多个目标。

案例

自动打开手机浏览器



示例代码：

```
Step("预置条件1: 回到桌面")  
self.driver.swipe_to_home()
```

```
Step("步骤1: 点击浏览器")
```

```
# 使用前先导入路径获取包
```

```
# from devicetest.utils.file_util import get_resource_path  
icon_res_path = get_resource_path(r'images/img.png')  
self.driver.touch_image(icon_res_path)
```

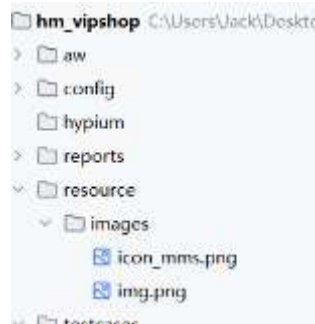
```
time.sleep(3)
```


总结

1、没有可以用于定位控件的唯一属性时，如何定位控件？

- **图片定位控件：**

- 前置：安装opencv-python包（pip install opencv-python）
- 使用：self.driver.touch_image(图片路径)
- 注意：图片保存位置为 项目根目录/resource/images



- 获取路径时先导包：from devicetest.utils.file_util
import get_resource_path
- 图片路径示例：get_resource_path(r'images/img.png')



比例坐标定位控件

- 当图像特征不明显，使用图像匹配方式无法准确识别控件位置，可以通过比例坐标的方式点击控件。

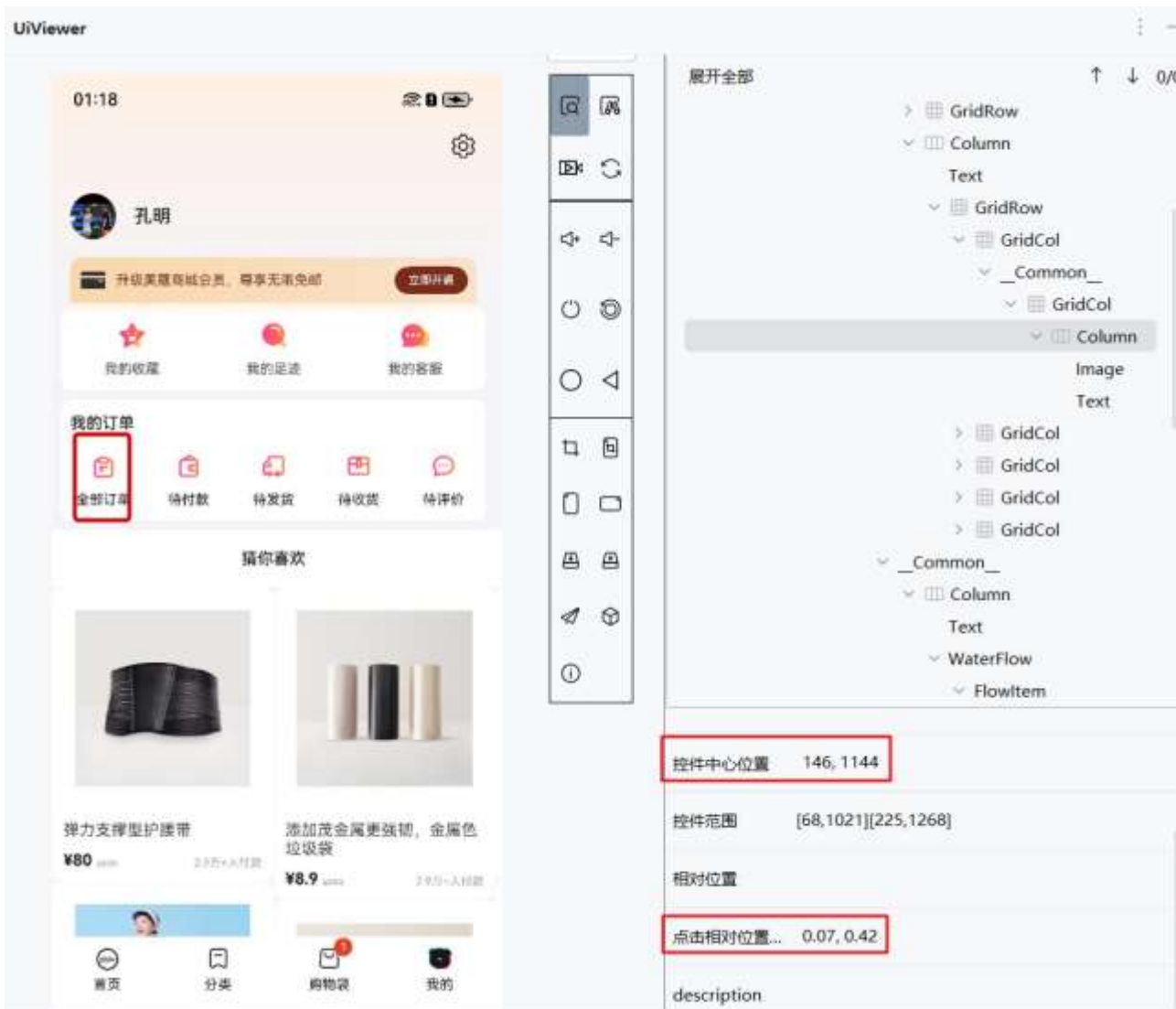


- 比例坐标可通过DevEco Testing Hypium插件查看。

点击相对位置范围 0.52, 0.98

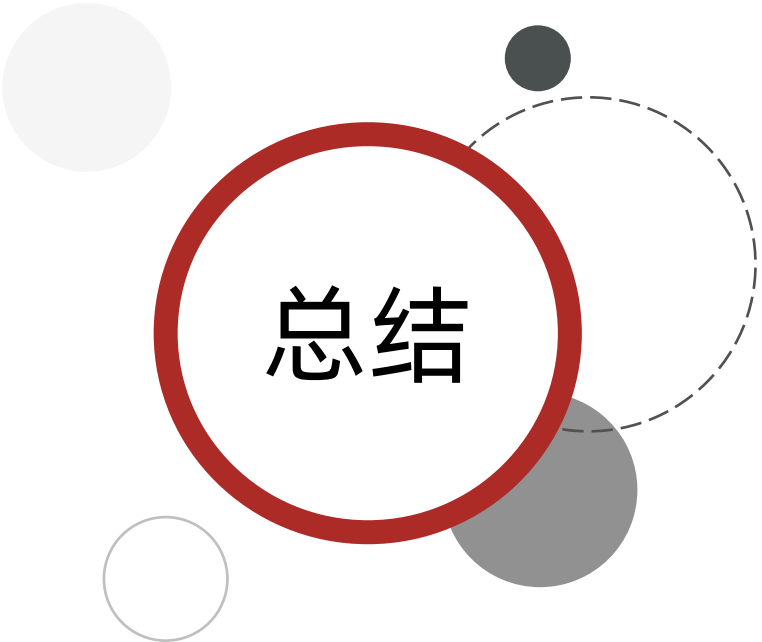
- 定位方法: `driver.touch((0.52, 0.98))` # 点击屏幕上(0.52 * 屏幕宽度, 0.98 * 屏幕高度)的位置

案例 查看全部订单信息



示例代码：

```
self.driver.touch((146, 1144))  
self.driver.touch((0.07, 0.42))
```



总结

1、当控件属性不唯一且图像特征不明显时，如何定位控件？

- **比例坐标定位：** `driver.touch((屏幕宽度, 屏幕高度))`
- 注意：该方式 **不推荐** 使用，因为在屏幕比例或者控件位置发生变化时该种方式 通常会失效，无法操作到正确的控件，并且如果实际操作界面不在当前界面，点击也会成功，但实际效果不符合预期



项目核心业务自动化

案例

下单业务自动化

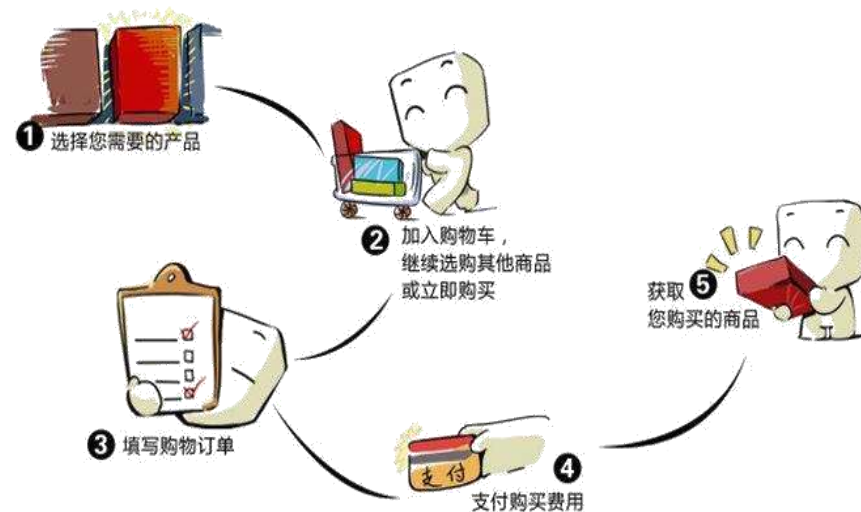
分析：

1、前置条件

- 获取商城app包名和界面名
- 启动商城程序
- 登录成功

2、操作步骤

- 搜索（查找搜索框==》输入关键字搜索商品==》查看搜索商品）
- 加入购物车（点击页面加入购物车==》选择商品规格==》再次点击加入购物车==》点击查看购物车）
- 支付（点击结算==》点击提交订单）





其他API

序号	操作	API	示例
01	查找窗口	find_window()	driver.find_window(WindowFilter().title("日历"))
02	触摸点击	touch()	driver.touch(BY.text("hello"))
03	多指点击	multi_finger_touch()	driver.multi_finger_touch([(0.1, 0.2), (0.3, 0.4)])
04	不太精准的滑动	swipe()	driver.swipe UiParam.UP, distance=40)
05	精准的滑动操作	slide()	driver.slide((100, 200), (300, 400))
06	拖拽	drag()	driver.drag(BY.text("文件.txt"), BY.text("上传文件"))
07	捏合缩小	pinch_in()	driver.pinch_in(BY.type("Image"))
08	双指放大	pinch_out()	driver.pinch_out(BY.type("Image"))
09	双指滑动	two_finger_swipe()	driver.two_finger_swipe((0.4, 0.4), (0.2, 0.2), (0.6, 0.6), (0.8, 0.8))
10	鼠标点击	mouse_click()	driver.mouse_long_click(BY.text("确认"), MouseButton.MOUSE_BUTTON_RIGHT)
11	鼠标拖拽	mouse_drag()	driver.mouse_drag(BY.text("控件1"), BY.text("控件2"))
12	鼠标移动	mouse_move()	driver.mouse_move(BY.text("控件1"), BY.text("控件2"))

序号	操作	API	示例
13	按键	press_key()	driver.press_key(KeyCode.POWER)
14	按组合键	press_combination_key()	driver.press_combination_key(KeyCode.CTRL_LEFT, KeyCode.SHIFT_LEFT, KeyCode.F)
15	hdcl命令执行	hdc()	driver.hdc("list targets")
16	shell命令执行	shell()	driver.shell("ls -l")
17	安装应用	install_app()	driver.install_app(r"test.hap")
18	卸载App	uninstall_app()	driver.uninstall_app(driver, "com.ohos.devicetest")
19	清除应用缓存数据	clear_app_data()	driver.clear_app_data("com.tencent.mm")
20	启动应用	start_app()	driver.start_app("com.huawei.hmos.browser", "MainAbility")
21	停止应用	stop_app()	driver.stop_app("com.ohos.settings")
22	拉取文件	pull_file()	driver.pull_file("/data/local/tmp/test.log", "test.log")
23	推送文件	push_file()	driver.push_file("test.hap", "/data/local/tmp/test.hap")
24	查询文件是否存在	has_file()	driver.has_file("/data/local/tmp/test_file.txt")

序号	操作	API	示例
25	检查是否相等	check_equal()	host.check(a, b, "a != b")
26	检查是否超出预期	check_greater()	host.check_greater(a, b)
27	检查控件是否存在	check_component_exist()	driver.check_component_exist(BY.type("Button"))
28	检查控件属性	check_component()	driver.check_component(BY.key("xxx"), checked=True)
29	检查图片是否存在	check_image_exist()	driver.check_image_exist("test.jpeg")
30	检查窗口	check_window()	driver.check_window(WindowFilter().focused(True), bundle_name="com.ohos.settings")

详细介绍参考官方文档

<https://developer.huawei.com/consumer/cn/doc/harmonyos-guides-V5/hypium-python-guidelines-V5>



传智教育旗下高端IT教育品牌